

Ecrire la fonction dump

- Ecrire la fonction Dump destinée à afficher sur le standard de sortie le contenu de la zone mémoire identifiée par le pointeur passé en paramètre.
- Vous tiendrez compte des contraintes définies par le cas d'utilisation proposé.

Le code:

```
cout << "---buff---";  
int buff[]={12,25,8,6,586};  
Dump(buff,5);  
  
cout<< endl << "---buff2---";  
int *buff2= new int[2];  
buff2[0]=45;  
buff2[1]=100;  
dump(buff2,2);  
delete buff2;
```

Donne à l'exécution:

```
---buff---  
12  
25  
8  
6  
586  
---buff2---  
45  
100
```

Ecrire la fonction Dump

```
#include <iostream>
using namespace std;

void dump(int* buff, unsigned int buffSize)
{
    for(unsigned int i=0; i<buffSize; ++i)
        cout << endl << buff[i];
}
```

Ecrire la classe Pen

- Cette classe modélise un stylo.
- Son emplacement est décrit par deux entiers x et y.
- Son état est décrit par un bool state (down->true, up->false)

Le code:

```
Pen p1 ; //p1 located at 0,0 ; p1 is up
```

```
Pen p2 (10,20) ; //p2 located at 10,20 ; p1 is up
```

```
p2.down();
```

```
p2.move(100,20);
```

```
cout << "pen p2->["
```

```
cout << p2.getState() << ', ';
```

```
cout << p2.getX() << ', ' << p2.getY(); << '];
```

```
p2.Up() ;
```

```
cout << endl << "pen p2->["
```

```
cout << p2.getState() << ', ';
```

```
cout << p2.getX() << ', ' << p2.getY(); << '];
```

Donne à l'exécution:

```
Pen p2->[1, 110, 40]
```

```
Pen p2->[0, 110, 40]
```

Ecrire la classe Pen

```
#ifndef PEN_H
#define PEN_H

class Pen
{
public:
    Pen();
    Pen(int, int);
    virtual ~Pen() = default;

    void move (int, int);
    void up();
    void down();
    int getX() const;
    int getY() const;
    bool getState() const;

private:
    int x,y;
    bool state;
};

#endif
```

```
#include "pen.h"

Pen::Pen():
x(0), y(0), state(false)
{
}

Pen::Pen(int _x, int _y):
x(_x), y(_y), state(false)
{
}

void Pen::move (int dx, int dy)
{
    x+=dx;
    y+=dy;
}

void Pen::up()
{
    state = false;
}

void Pen::down()
{
    state=true;
}

int Pen::getX() const
{
    return x;
}

int Pen::getY() const
{
    return y;
}

bool Pen::getState() const
{
    return state;
}
```

Ecrire la classe Complex

- Cette classe modélise un nombre complexe possédant deux données membres de type double nommées: re et im;
- La fonction double sqrt(double) est supposée exister.
 - racine carrée.
- Le constructeur par défaut est interdit.

Le code:

```
Complex c1(10,20);
```

```
Complex c2(2,2);
```

```
Complex c3=100;
```

```
c3.printOn(cout);
```

```
(c1+c2).printOn(cout << endl);
```

```
cout << endl << c2.Module();
```

```
double module=c2;
```

```
cout << endl << module;
```

Donne à l'exécution:

```
100 +i 0
```

```
12 +i 22
```

```
2.82843
```

```
2.82843
```

Classe Complex

```
• #ifndef COMPLEX_H
• #define COMPLEX_H

• class Complex
• {
• public:
•     Complex(double re, double im=0);
•     virtual ~Complex() = default;

•     Complex operator+(const Complex&) const;
•     operator double()const;
•     double module()const;

•     ostream& Complex::printOn(ostream&) const

• private:
•     double re, im;
• };

• #endif
```

```
#include "complex.h"

Complex::Complex(double _re, double _im):
re(_re), im(_im)
{
}

ostream& Complex::printOn(ostream& os) const
{
    return os << re<< " +i "<< im;
}

Complex Complex::operator +(const Complex& c)
const
{
    return Complex(re+c.re, im+c.im);
}

double Complex::module()const
{
    return sqrt(re*re+im*im);
}

Complex:: operator double()const
{
    return module();
}
```

Ecrire les classes Geometric et Line

- La classe Geometric:
 - elle est abstraite,
 - elle définit le comportement virtuel pure Move (déplacement relatif) prenant en paramètre deux entiers;
 - elle doit permettre le mécanisme d'impression pour ses classes héritières.
- La classe Line
 - elle est un objet géométrique et modélise une ligne possédant deux données membres de type Point nommées: start et end;
- La classe Point est supposée exister.
 - elle hérite de la classe Geometric.
- La classe Base est supposée exister
 - elle fournit le mécanisme d'impression.

Le code:

```
Line l1;  
Line l2(Point(), Point(10, 50));  
cout << endl << "l1: " << l1;  
cout << endl << "l2: " << l2;  
l2.Move(5,10);  
cout << endl << "l2: " << l2;  
cout << endl << "l2 start: " << l2.getStart();  
cout << endl << "l2 end: " << l2.getEnd();
```

Donne à l'exécution:

```
l1: line(Point(0, 0), Point(0, 0))  
l2: line(Point(0, 0), Point(10, 50))  
l2: line(Point(5, 10), Point(15, 60))  
l2 start: Point(5, 10)  
l2 end: Point(15, 60)
```

Classes Geometric et Line

```
• #ifndef GEOMETRIC_H
• #define GEOMETRIC_H

• #include "base.h"

• class Geometric :public Base{
• public:
•     virtual ~Geometric() = default;
•     virtual void move(int, int)=0;
• };
• #endif

• #ifndef LINE_H
• #define LINE_H

• #include "geometric.h"
• #include "point.h"

• class Line :public Geometric
• {
• public:
•     Line();
•     Line(const Point& , const Point&);
•     virtual ~Line() = default;

•     virtual void move(int, int);
•     Point getStart() const;
•     Point getEnd() const;

• protected:
•     ostream& PrintOn(ostream&) const;

• private:
•     Point start, end;
```

```
#include "line.h"

Line::Line(const Point& _start, const Point& _end):
start(_start), end(_end)
{
}

void Line::move(int dx, int dy)
{
    start.move(dx, dy);
    end.move(dx, dy);
}

Point Line::getStart() const
{
    return start;
}

Point Line::getEnd() const
{
    return end;
}

ostream& Line::printOn(ostream& os )const
{
    os << "line(" << start << ',' << end << ')';
    return os;
}
```


Ecrire la classe générique Value

- La classe Value:
 - elle est générique,
 - elle possède une donnée membre de type T appelée value.
 - elle doit pouvoir s'imprimer.
- Ecrire l'opérateur global / permettant la division entre Value<T> et fournissant en résultat un Value<T>
 - On considèrera que pour son usage particulier le type T supporte l'opérateur /.
- La classe Base est supposée exister
 - elle fournit le mécanisme d'impression.
- La fonction string value2string(double) est supposée exister.
 - convertit un double en une chaîne de caractères.

Le code:

```
Value<int> vint1(9),vint2(2), vint4(0);
Value<string> vstring("hello");
try
{
    cout << "value of vint1: "<<vint1.getValue();
    vint1.setValue(10);
    cout << endl << vint1;
    cout << endl << vstring;
    cout << endl << vint1/vint2;
    cout << endl << vint1/vint4;
}
catch(const char* ex)
{
    cout << endl << ex;
}
```

Donne à l'exécution:

```
value of vint1: 9
value(10)
value(hello)
value(5)
division by zero
```

Classes Value

```
#ifndef VALUE_H
#define VALUE_H

#include "base.h"

template <class T>
class Value :public Base
{
public:
    Value(const T& v=T());
    virtual ~Value() = default;

    T GetValue() const;
    void SetValue(const T&);

    virtual ostream& Complex::PrintOn(ostream& os) const;

private:
    T value;
};

//operator /

template<class T>
Value<T> operator /(const Value<T>& left, const Value<T>&
    right)
{
    if(right.GetValue()==0)
        throw "division by zero";
    return Value<T>(left.GetValue()/right.GetValue());
}
```

```
template<class T>
Value<T>::Value(const T& v):
value(v)
{
}

template<class T>
T Value<T>::GetValue() const
{
    return value;
}

template<class T>
void Value<T>::setValue(const T& v)
{
    value=v;
}

template <class T>
virtual ostream& Complex::printOn(ostream& os) const
{
    return os << "value(" << value << ')';
}
```